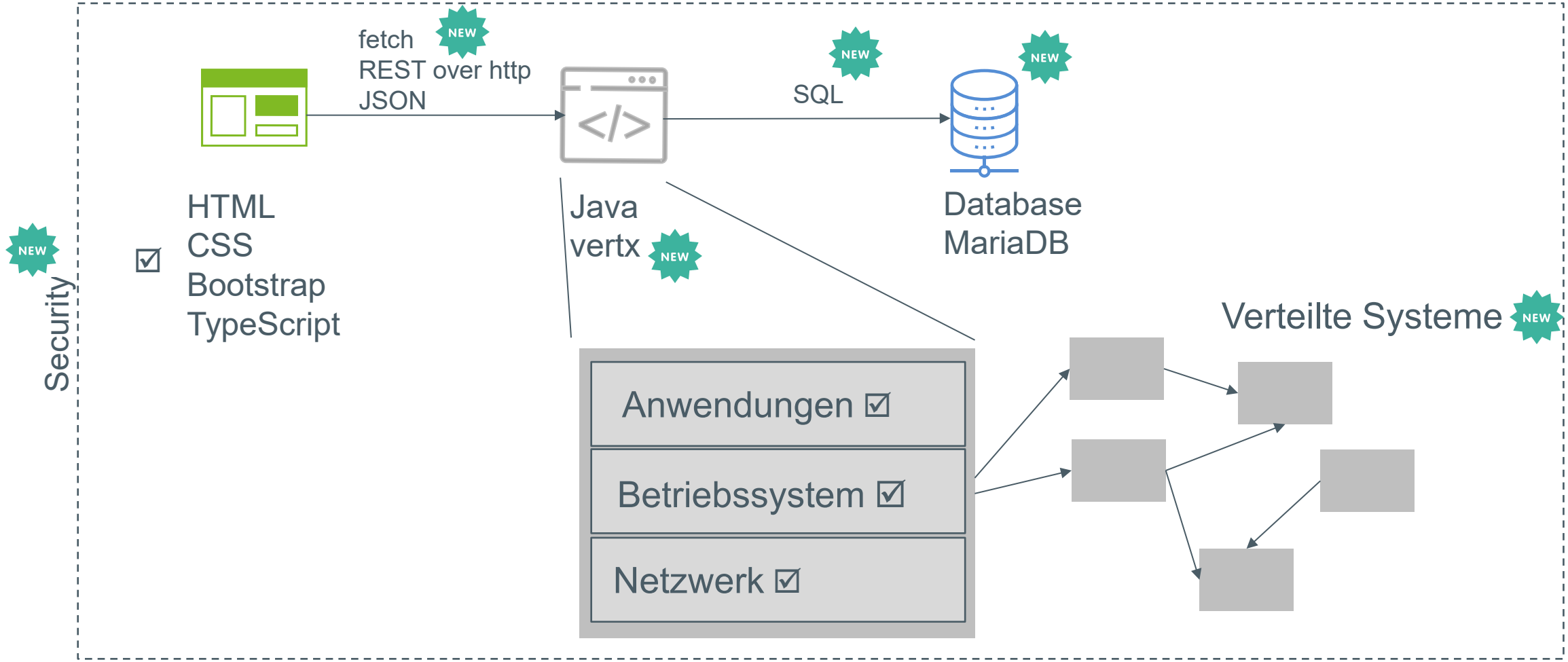
Agenda

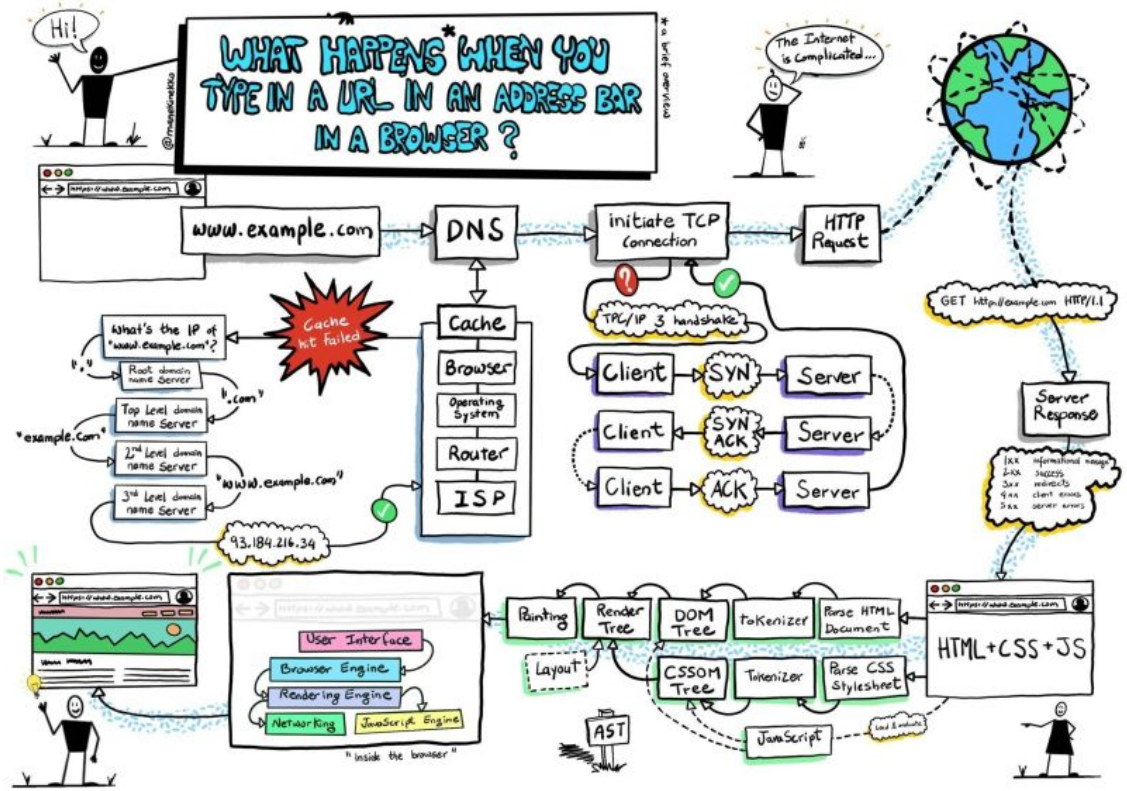
1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	**Verteilte Systeme**	**102**
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Das Ziel von PI 2



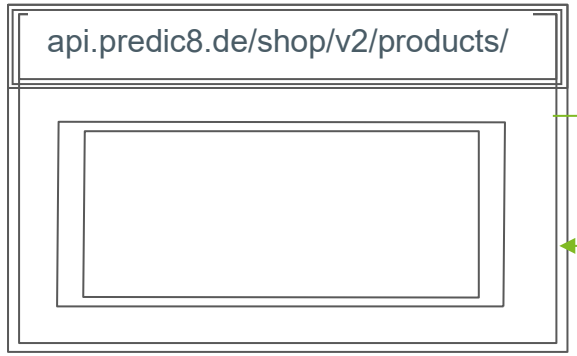
Motivation

- Bisher hatten wir nur statischen Inhalt (HTML + CSS), der in Dateien stand.
- Wir wollen jetzt dynamisch Inhalt erzeugen, der zum Beispiel aus Datenbanken kommt.

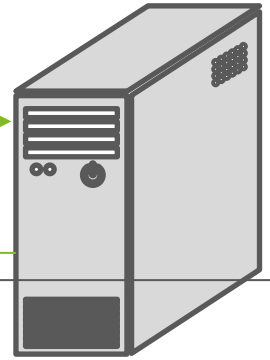


Hinter der URL muss also auf dem Server ein Programm stehen, das auf einen Request antwortet.

Beispiel




```
GET /shop/v2/products/ HTTP/1.1
Host: api.predic8.de
…
```



```
HTTP/1.1 200 OK
< Content-Type: application/json
{
        "meta": {"count": 18,
                 "start": 1,
                 "limit": 10,
                 "next_link": "/shop/v2/products?start=11&limit=10"
        },
        "products": [
                 { "id": 18,
                   "name": "Melone",
                   "self_link": "/shop/v2/products/18"
                 } …
                 ]
}
```

JSON
JavaScript Object Notation

Wir kennen JSON bereits

Letztes Semester …

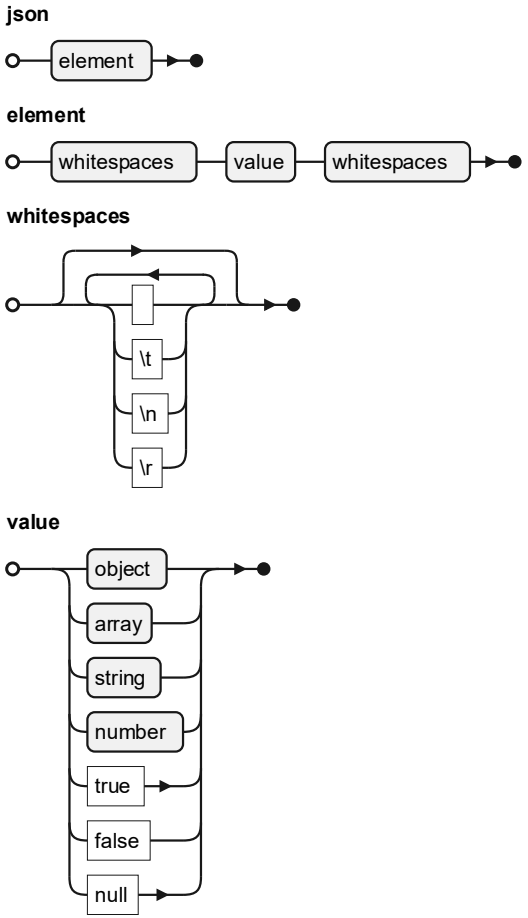
- Curl https://api.predic8.de/shop/v2/products


```
{"meta":{"count":19,"start":1,"limit":10,"next_link":"/shop/v2/products?start=11&limit=10&sort=id&order=asc"},"products":[{"id":1,"name":"Banana","self_link":"/shop/v2/products/1"},{"id":2,"name":"Blackberry","self_link":"/shop/v2/products/2"},{"id":3,"name":"Cherry","self_link":"/shop/v2/products/3"},{"id":4,"name":"Coconut","self_link":"/shop/v2/products/4"},{"id":5,"name":"Dragon-Fruit","self_link":"/shop/v2/products/5"},{"id":6,"name":"Fig","self_link":"/shop/v2/products/6"},{"id":7,"name":"Gac-Fruit","self_link":"/shop/v2/products/7"},{"id":8,"name":"Grapes","self_link":"/shop/v2/products/8"},{"id":9,"name":"Horn-Cucumber","self_link":"/shop/v2/products/9"},{"id":10,"name":"Lychee","self_link":"/shop/v2/products/10"}]}
```


TypeScript Konfiguration: tsconfig.json

```
{
  "compilerOptions": {
    "target": "es2016",
    "module": "commonjs",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true,
    "outDir": "build"
  }
}
```

JSON Syntax



Beispiele:

Object:	{ }
Array:	[ ]
String:	"Hello World"
Number:	42
	1.23
Boolean:	true
	false
Null:	null

} Ermöglichen mehrere Werte

☺ Einfaches Format

☹ Keine komplexen Datentypen (z.B. Datum)

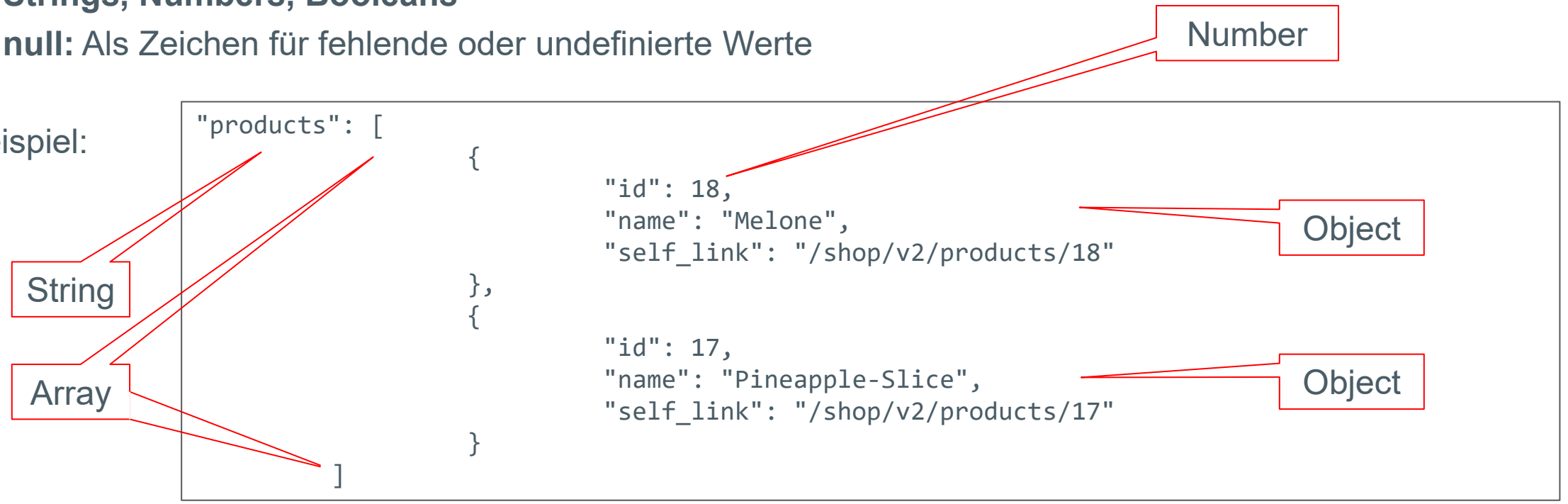
☹ Es gibt in JSON keine Kommentare

JSON - Struktur

Eine JSON-Datei (oder –Nachricht) ist eine baumartige Struktur, die also aus folgenden Elementen besteht:

- **Objekte:** Objekte sind (mehrere) Schlüssel-Wert Paare.
- **Arrays:** Geordnete Sammlungen von Werten (z.B. Objekten)
- **Strings, Numbers, Booleans**
- **null:** Als Zeichen für fehlende oder undefinierte Werte

Beispiel:



JSON - Zusammenfassung



- JSON wurde 2001 von Douglas Crockford erfunden

Vorteile:

- JSON ist für Menschen gut lesbar
- JSON ist für Maschinen gut zu verarbeiten
- JSON ist, da es nur reinen Text erfordert, überall nutzbar

Nachteil: Es gibt in JSON keine Kommentare.

Einsatzgebiete:

- Datenaustausch zwischen Client und Server
- Umwandeln von JavaScript-Objekten in eine textuelle Darstellung und zurück („Serialisieren“ und „Deserialisieren“).

Routen

Zu jeder Route eine Funktion hinterlegen,
die ausgeführt wird, wenn die URL aufgerufen wird.



Daten an den Server senden per Node Express („Web Service“)

```
import express, { Express, Request, Response } from
'express';

let names : string[] = [];
const app: Express = express();
const port:number = 3000;
app.use(express.json());

app.get('/', (req: Request, res: Response) => {
   res.send('Moin!');
});

app.post('/names', (req:Request, res:Response):void => {
   console.log(req.body);
   const name:string =
req.body.vorname+req.body.nachname;
   names.push(name)
   res.json({"name" : name});
});

app.get('/names', (req: Request, res: Response) => {
   res.send(names);
});
```

Ggf. vorher mit `npm install express` installieren

JSON-Parser

```
POST localhost:3000/names
{
  "vorname": "Paul",
  "nachname": "Willi"
}
```

→

```
{
  "name": "PaulWilli"
}
```

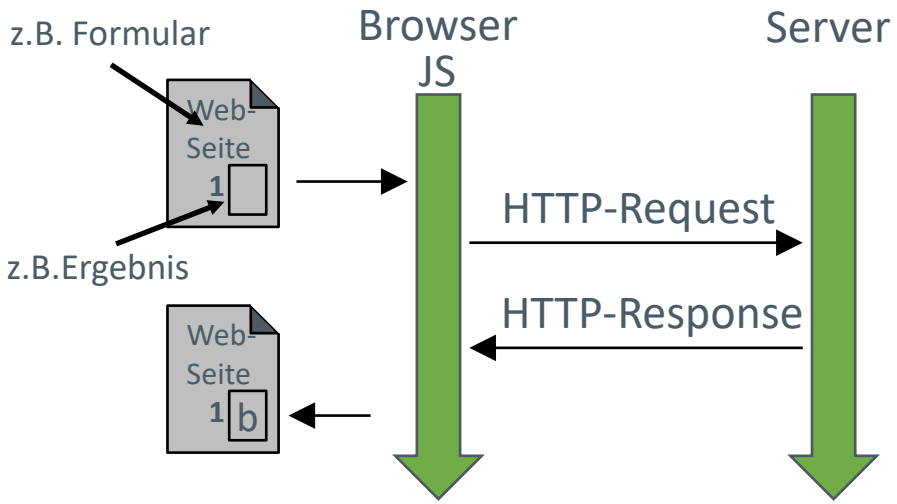
```
GET localhost:3000/names
```

→

```
{
  "names": [
    "PaulWilli",
    "HeinzOtto"
  ]
}
```

Wozu Services aufrufen?

- Wir wollen Web Services nutzen, um HTML-Seiten im Browser zu aktualisieren, ohne dass wir eine komplett neue Seite laden müssen
- Die Kommunikation ist asynchron, d.h. der Browser bzw. die HTML-Seite fragt einen Server an und arbeitet weiter
- Der Server sendet irgendwann eine Antwort zurück, die im Browser benutzt werden kann, um das DOM zu ändern.



Fetch API

Die **Fetch API** ist eine Schnittstelle zum Abrufen von Ressourcen

- Generische Definition von Request/Response und ist so multifunktional überall einsetzbar, wo mit Anfragen und Antworten gearbeitet wird
- Da man mit der `fetch-Methode` jede HTTP-Request-Methode aufrufen kann, ist sie sehr gut zur Umsetzung von REST-Requests (nächstes Kapitel …) geeignet.
- `fetch()` nimmt den Pfad einer Ressource als Argument entgegen, ist global verfügbar und gibt ein `Promise` zurück.

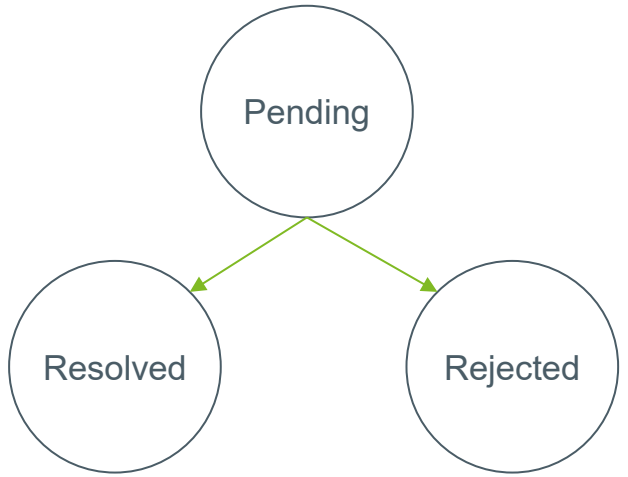
Voraussetzung:

- Es muss eine `tsconfig.json` existieren
  (wenn nicht vorhanden: File -> New -> tsconfig.json File)
- Target-Feld in der `tsconfig.json` muss auf „`es6`“ stehen
- Eine Funktion, in der `fetch` benutzt wird, muss mit „`async`“ markiert werden
  - Um dem Compiler zu sagen „Hier wird eine Operation stattfinden, die eine unbestimmte Zeit dauern kann“

https://developer.mozilla.org/de/docs/Web/API/Fetch_API

Häh? Promise?

- Promises sind ein mächtiges Konstrukt, um asynchrone Operationen zu behandeln (wie zum Beispiel das Lesen von Daten von einem Service).
- Ein Promise ist ein Objekt, das die zukünftige Erfüllung oder Ablehnung einer asynchronen Operation repräsentiert. Es hat drei mögliche Zustände:
  - Ausstehend (Pending): Anfangszustand, weder erfüllt noch abgelehnt.
  - Erfüllt (Gelöst, Resolved): Die Operation wurde erfolgreich abgeschlossen, und ein Ergebnis steht zur Verfügung.
  - Abgelehnt (Rejected): Die Operation hat einen Fehler erlitten, und ein Fehlergrund steht zur Verfügung.



- Der Promise-Konstruktor nimmt eine Funktion mit zwei Parametern entgegen: `resolve` und `reject`. In dieser Funktion führen Sie die asynchrone Operation aus und rufen `resolve`(Ergebnis) auf, wenn sie erfolgreich ist, oder `reject`(Fehler) auf, wenn sie fehlschlägt.

Beispiel: Addierer (Backend)

Adder.ts

```
import express, { Express, Request, Response } from 'express';

const app : Express = express();
const port : number = 3000;

app.use(express.json());
app.use("/", express.static(__dirname));

app.listen(port, () => {console.log('server running‘) });

app.post("/add", (req: Request, res: Response) => {
   let reply : number = Number(req.body.a) + Number(req.body.b)
+                       Number(req.body.c);
   res.status(200) .end(JSON.stringify({sum:reply}));

});
```

Statische Route

Dynamische Route /add (POST)

Ergebnis als JSON zurückgeben Beispiel: {“sum“ : 45}

Beispiel: Addierer (HTML)

Adder.html

```
<script src=adderFetch.js></script>
…
<div class="container">
   <h1>Adder</h1>
   <form id="add-form">
      <div class="input-group">
         <span class="input-group-text">a</span>
         <input type="text" id="a" class="form-control">
         <span class="input-group-text">b</span>
         <input type="text" id="b" class="form-control">
         <span class="input-group-text">c</span>
         <input type="text" id="c" class="form-control">
         <button class="btn btn-outline-secondary" type="submit"> =  </button>
         <span class="input-group-text" id="result">?</span>
      </div>
   </form>
```

Beispiel: Addierer (Front-End)

adderFetch.ts

```
document.addEventListener('DOMContentLoaded', () => {
   document.getElementById('add-form')?.addEventListener('submit', async (event) => {
      event.preventDefault();
      const a = (document.getElementById('a') as HTMLInputElement).value;
      const b = (document.getElementById('b') as HTMLInputElement).value;
      const c = (document.getElementById('c') as HTMLInputElement).value;
      const res = await fetch('add', {
         method: 'post',
         headers: {
            "Content-type": "application/json"
         },
         body: JSON.stringify({a, b, c})
      });
      const data = await res.json();
      console.log(res.status + ' ' + res.statusText);
      if (res.ok) {
         const resul = document.getElementById('result');
         if (resul) {
            resul.textContent = data.sum;
         }
      }
   });
});
```

Fetch-Aufruf

Warte auf die Antwort (asynchron)

Antwort in DOM einfügen

Recap: Fetch API

Die Fetch API ist eine Schnittstelle zum Abrufen von Ressourcen

- Generische Definition von Request/Response und ist so multifunktional überall einsetzbar, wo mit Anfragen und Antworten gearbeitet wird
- Da man mit der `fetch-Methode` jede HTTP-Request-Methode aufrufen kann, ist sie sehr gut zur Umsetzung von REST (nächstes Kapitel …) geeignet.
- `fetch()` nimmt den Pfad einer Ressource als Argument entgegen, ist global verfügbar und gibt eine `Promise` zurück.

Voraussetzung:

- Es muss eine `tsconfig.json` existieren
  (wenn nicht vorhanden: File -> New -> tsconfig.json File)
- Target-Feld in der `tsconfig.json` muss auf „`es6`“ stehen
- Eine Funktion, in der `fetch` benutzt wird, muss mit „`async`“ markiert werden
  - Um dem Compiler zu sagen „Hier wird eine Operation stattfinden, die eine unbestimmte Zeit dauern kann“

https://developer.mozilla.org/de/docs/Web/API/Fetch_API